

1 TSP 问题

本次作业旨在求解旅行商问题 (Traveling Salesperson Problem, TSP)。问题设定为在 100×100 的二维平面内随机生成 30 个城市节点。求解目标是寻找一条遍历所有城市且每个城市仅访问一次的最短闭合回路。通过随机生成二维坐标生成城市的分布如图 1 所示。

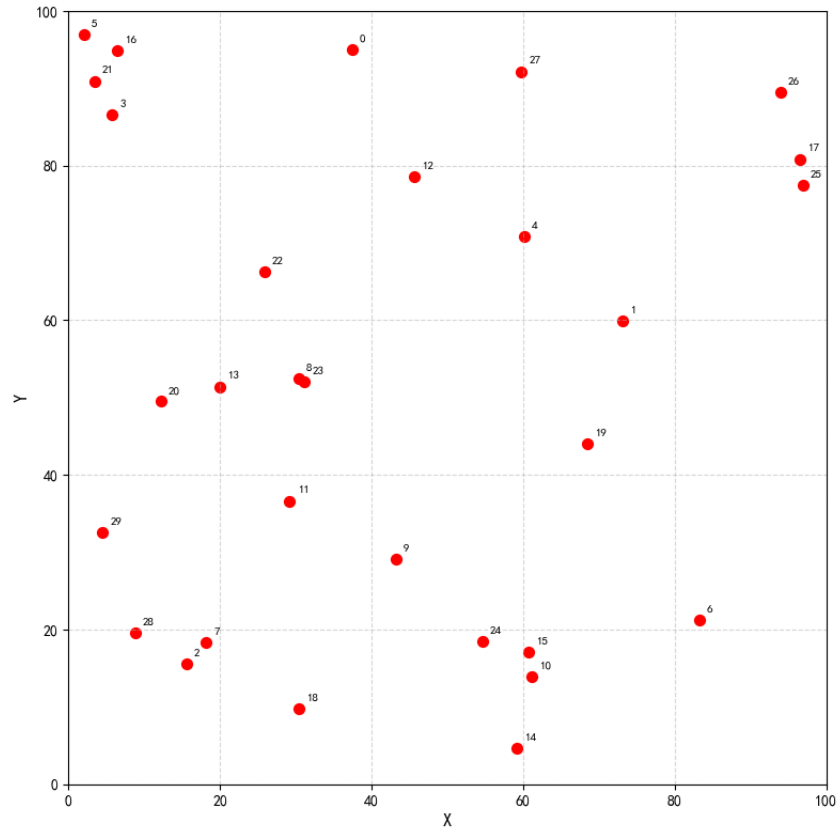


图 1: TSP 问题 30 个城市的分布

2 模拟退火算法 (Simulated Annealing, SA)

2.1 算法实现

对于新路径的生成, 采用 2-opt 策略 (即逆序操作)。在当前路径中随机选择两个切分点, 将中间部分的路径片段反转, 以此产生新解。

本算法采用了两种不同的停止准则相结合。除了设置最低温度阈值 (当 $T < T_{\min}$ 则停止) 外, 如果算法连续若干次降温过程中都未接受任何新解, 算法也将提前停止。

代码主体部分如下所示:

```
while T > T_min and frozen_count <= frozen:
    accepted = 0
    for _ in range(iter):
        new_route = _new_route(route)
        new_dist = _route_dist(new_route, dist_mat)
        delta = new_dist - dist # dE
        # 接受准则
```

```

if delta < 0:
    route = new_route
    dist = new_dist
    accepted += 1
    if dist < best_dist:
        best_dist = dist # 更新全局最优解
        best_route = route.copy()
else:
    prob = math.exp(-delta / T) # 以一定概率接受
    if np.random.rand() < prob:
        route = new_route
        dist = new_dist
        accepted += 1
history.append(best_dist) # 当前温度下的最优解
T *= alpha
if accepted == 0:
    frozen_count += 1
else:
    frozen_count = 0

```

2.2 参数调整

对于初始温度的选取, 尝试了课件上的方法。在正式退火前, 随机生成一批样本计算其路径长度, 求出最大差值 $\Delta f_{\max} = f_{\max} - f_{\min}$, 并根据期望的初始接受概率 P_0 反推初始温度 $T_0 = -\Delta f_{\max} / \ln(P_0)$ 。如果取样本数量为 100, $P_0 = 0.9$, 计算出的初始温度高达 5566.37, 但事实上完全不需要这么高的初始温度。

为了寻找模拟退火算法的最佳参数, 我们采用网格搜索的策略, 对初始温度 T_0 、降温系数 α 以及每个温度下的迭代次数 $iter$ 进行了遍历。

经过测试, 确定的最佳参数组合为: $T_0 = 50$, 降温系数 $\alpha = 0.95$, 同一温度下迭代次数 $iter = 50$ 。

网格搜索最优的八个参数组合如表 1 所示, 其中前七个组合均能找到最短路径长度为 451.77 的解。

表 1: SA 算法前八个最优参数组合

T_0	α	iter	最短距离	耗时 (秒)
50	0.95	50	451.77	0.1070
200	0.99	10	451.77	0.1330
50	0.95	100	451.77	0.2130
5	0.99	100	451.77	0.6721
500	0.99	50	451.77	0.7556
200	0.99	100	451.77	1.2712
500	0.99	100	451.77	1.3855
5000	0.95	50	453.90	0.1678

网格搜索最差的八个参数组合如表 2 所示, 其中当初始温度 $T_0 = 5$, 降温系数 $\alpha = 0.9$, 同一温度下迭代次数 $iter = 10$ 时, 算法仅耗时 0.009 秒, 最终得到的解为 621.27, 说明这样的参数组合几乎没有对解空间作细致的搜索。

表 2: SA 算法后八个最差参数组合

T_0	α	iter	最短距离	耗时 (秒)
5000	0.95	10	508.44	0.0352
500	0.95	10	517.36	0.0298
5	0.95	10	536.81	0.0130
200	0.90	10	550.07	0.0180
500	0.90	10	550.75	0.0130
50	0.90	10	559.15	0.0129
5000	0.90	10	591.40	0.0160
5	0.90	10	621.27	0.0090

通过对网格搜索的结果进行分析,可以发现:降温系数 α 和同一温度下的迭代次数 $iter$ 是决定解质量的关键参数。

当 α 较小(如 0.90)时,系统降温过快,容易在算法早期丧失跳出局部最优的能力,导致最终生成的路径存在“交叉”现象,解的质量较差。而当采用较慢的速度退火(如 $\alpha = 0.99$)时,算法能在解空间中进行充分探索,稳定收敛至全局最优解。

观察表 2 可以发现,最差的八个参数组合中, $iter$ 都仅仅取了 10 次。在任何一个固定的温度 T 下,模拟退火算法寻找新解的过程本质上是一个马尔可夫链蒙特卡洛(MCMC)过程,遵循 Metropolis 接受准则:

$$P = \exp\left(-\frac{\Delta E}{T}\right)$$

其中 ΔE 是新解与当前解之间的路径长度差。只有当状态转移的次数足够多时,马尔科夫链才能收敛到平稳分布(即玻尔兹曼分布)。若迭代次数过少(如 $iter = 10$),系统采样的样本不足以代表该温度下的真实概率分布,算法可能偏离全局最优解。另一个角度看,较大的 $iter$ 使得算法在每个温度下能够充分地探索解空间,增加了接受更优解的机会,从而提高了最终解的质量。

而对于初始温度 T_0 的选取,取较高的初始温度本意为使算法在初始阶段具有较高的接受概率,以便在解空间中进行充分的探索,即 $\Pr\{\bar{E} = E(r)\} \approx \frac{1}{|D|}, T \rightarrow \infty$ 。但实验结果表明,即使取较低的初始温度 $T_0 = 5$ 时,辅以较大的降温系数 $\alpha = 0.99$ 和较大的迭代次数 $iter = 100$,算法仍能达到全局最优解。

2.3 结果可视化

SA 算法生成的最短路径如图 2 所示,是一条没有交叉点的环路。SA 算法最优解的迭代曲线如图 3 所示,当迭代次数为 109、温度为 0.2 时,算法已经达到全局最优解。

4. 指数排序: 根据适应度对个体进行排序, 并根据排名赋予选择概率 $p_i = \frac{q^{rank_i-1}}{\sum_{j=1}^N q^{rank_j-1}}$, 其中 $rank$ 是个体的排名, q 取 0.99。代码中将其记为 “ranking”。

对于交叉的策略, 实现了顺序交叉 (OX) 和基于顺序的交叉 (OBX)。

1. OX: 随机截取父代 1 的一段基因给子代, 然后按父代 2 的顺序填补剩下的空缺。

2. OBX: 从父代 1 中随机选择一组城市, 在父代 2 中找到这些城市并清空, 然后按它们在父代 1 中的出现顺序, 依次填入父代 2 的空位中。

这两种方法均能保证父代的相对城市顺序得以遗传, 且生成的子代是合法的 TSP 路径。

对于变异的策略, 实现了单点交换变异与逆序变异。

1. 单点交换变异: 以概率 pm 随机交换两个城市的位置。代码中将其记为 “swap”。

2. 逆序变异: 以概率 pm 随机反转路径的某部分。代码中将其记为 “inverse”。

对于选择策略, 我们引入了精英保留策略, 即每次迭代保留上一代中适应度最高的个体, 以防止最优解丢失。

此外, 采用了早停策略, 当连续若干代最优解未更新时, 算法提前终止。算法主循环如下:

```
for gen in range(generations):
    distances = np.array([_route_dist(ind, dist_mat) for ind in population])
    fitness = 1.0 / (distances + 1e-6) # 距离的倒数作为适应度
    min_idx = np.argmin(distances)
    improved = False
    if distances[min_idx] < best_dist:
        best_dist = distances[min_idx]
        best_route = population[min_idx].copy()
        improved = True
    history.append(best_dist)

    # 早停: 最优解连续若干代不更新
    if improved:
        count = 0
    else:
        count += 1
        if (stop is not None) and (count >= stop):
            break

    if selection_method == 'truncation':
        selected = _truncation_selection(population, fitness, pop_size, T=0.5)
    elif selection_method == 'tournament':
        selected = _tournament_selection(population, fitness, pop_size, k=3)
    elif selection_method == 'ranking':
        selected = _ranking_selection(population, fitness, pop_size)
    elif selection_method == 'default':
        selected = _default_selection(population, fitness, pop_size)
    else:
        raise ValueError("No such selection method!")

    new_pop = [best_route.copy()] # 精英保留
```

```

for i in range(1, pop_size, 2):
    parent1 = selected[i]
    parent2 = selected[(i + 1) % pop_size]

    # 交叉
    if np.random.rand() < pc:
        if crossover_method == "ox":
            child1 = _ox_crossover(parent1, parent2)
            child2 = _ox_crossover(parent2, parent1)
        elif crossover_method == "obx":
            child1 = _obx_crossover(parent1, parent2)
            child2 = _obx_crossover(parent2, parent1)
        else:
            raise ValueError("No such crossover method!")
    else:
        child1, child2 = parent1.copy(), parent2.copy()

    # 变异
    if mutation_method == 'inverse':
        child1 = _inverse_mutate(child1, pm)
        child2 = _inverse_mutate(child2, pm)
    elif mutation_method == "swap":
        child1 = _swap_mutate(child1, pm)
        child2 = _swap_mutate(child2, pm)
    else:
        raise ValueError("No such mutation method!")

    new_pop.extend([child1, child2])

population = new_pop[:pop_size] # 保持种群规模不变, 截断多余的

```

3.2 参数调整

固定代数为 800, 对种群规模 pop_size 、交叉率 p_c 、变异率 p_m 以及不同组合的交叉/变异/选择策略进行了网格搜索。

最优参数组合为: 种群规模 $pop_size = 50$, $p_c = 0.9$, $p_m = 0.2$, 选择策略为锦标赛选择, 交叉策略为 OBX, 变异策略为逆序变异。

网格搜索最优的八个参数组合如表 3 所示, 均能找到最短路径长度为 451.77 的解。

表 3: GA 算法前八个最优参数组合

种群规模	交叉率	变异率	选择策略	交叉策略	变异策略	最短距离	耗时 (秒)
50	0.9	0.20	tournament	obx	inverse	451.77	3.741
100	0.7	0.05	truncation	obx	inverse	451.77	4.602
100	0.7	0.05	default	obx	inverse	451.77	5.567
100	0.9	0.05	tournament	obx	inverse	451.77	5.653
100	0.9	0.20	truncation	obx	inverse	451.77	6.315
100	0.9	0.05	tournament	ox	inverse	451.77	6.898
200	0.9	0.20	truncation	obx	inverse	451.77	11.730
200	0.9	0.05	tournament	obx	inverse	451.77	11.805

网格搜索最差的八个参数组合如表 4 所示, 其中最差的策略组合为轮盘赌选择、OX 交叉和单点交换变异, 仅能得到最短路径为 847.08 的解且耗时超过 10 秒。

表 4: GA 算法最差的八个参数组合

种群规模	交叉率	变异率	选择策略	交叉策略	变异策略	最短距离	耗时 (秒)
50	0.9	0.20	ranking	obx	inverse	741.43	3.669
200	0.9	0.05	default	ox	inverse	752.12	16.782
100	0.9	0.20	default	obx	swap	760.38	6.147
200	0.9	0.20	default	ox	inverse	775.14	16.575
100	0.9	0.05	default	ox	swap	808.20	8.372
50	0.9	0.05	ranking	obx	inverse	808.90	3.265
50	0.9	0.20	ranking	obx	swap	825.44	3.082
100	0.9	0.20	default	ox	swap	847.08	10.206

分析网格搜索的结果可以发现, 策略的选择对 TSP 问题具有较大的影响。

在表现优异的参数组合中, 逆序变异 (inverse) 占据了 100% 的比例。这是因为对路径的某一部分进行逆序操作有时可以看作解开一次路径的交叉 (即 2-opt 思想), 其寻找最优解的能力超过盲目的单点交换变异 (swap)。

OBX 交叉策略在前八个最优参数组合中占据了 87.5% 的比例, 这可能与它既保持了父代 1 的相对顺序和父代 2 的绝对顺序, 又能产生较多多样化的子代有关。

此外, 相对来说锦标赛选择与截断选择是更优的选择策略, 而最差的八个参数组合中都使用了轮盘赌选择 (default) 和排序选择 (ranking)。这可能是因为在锦标赛选择和截断选择中, 那些最优秀的个体更有可能被选中; 而轮盘赌选择和排序选择对适应度的分布较为敏感。

3.3 结果可视化

GA 算法生成的最短路径如图 4 所示, 其生成的最短路径与 SA 算法的结果完全一致。GA 算法最优解的迭代曲线如图 5 所示, 其在 102 代时已经达到全局最优解。

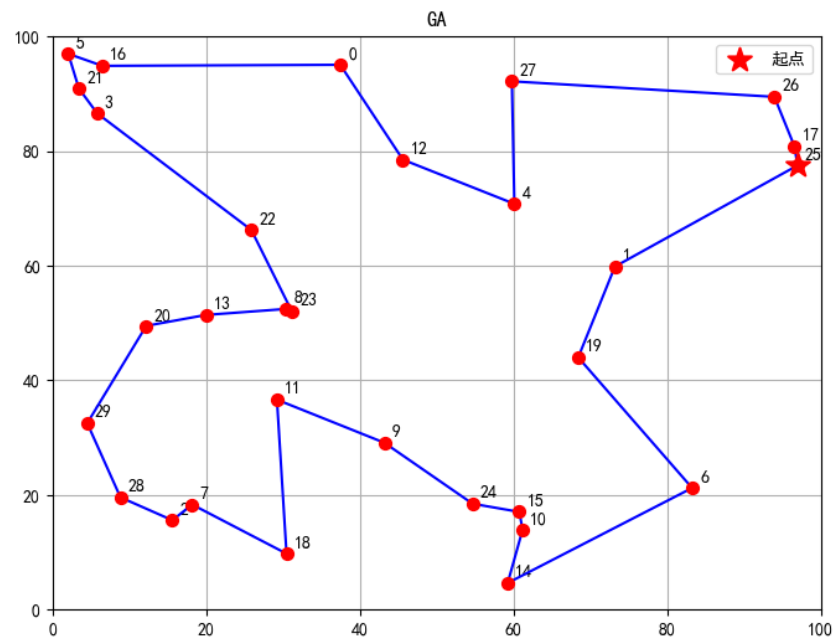


图 4: GA 算法生成的最短路径

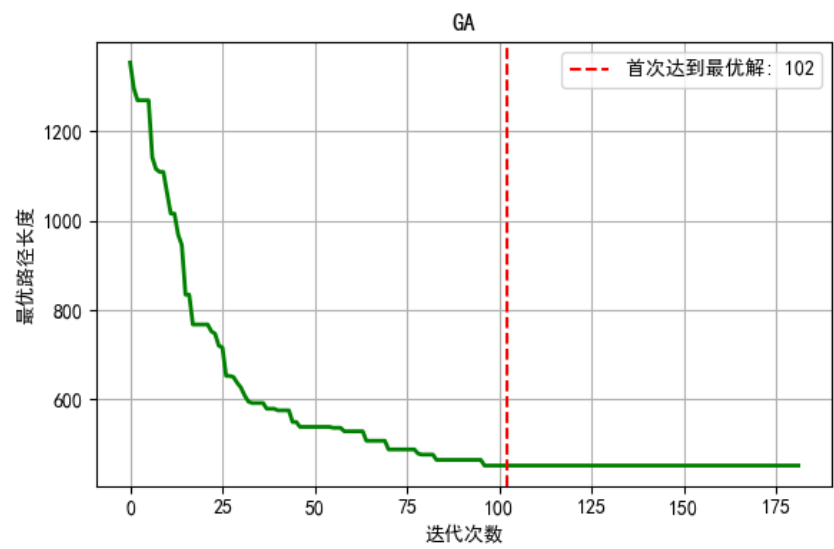


图 5: GA 算法最优解的迭代曲线

4 两种算法的对比

在得到两种算法的最优参数组合后，我们从最短距离和计算效率（时间）对模拟退火 (SA) 和遗传算法 (GA) 进行了对比，如表 5所示：

表 5: 对比 SA 算法和 GA 算法

算法	最短距离	耗时 (秒)
SA	451.77	0.107
GA	451.77	3.741

在参数调整得当的情况下, 模拟退火算法和遗传算法均能最终找到长度为 451.77 的最短路径。该解极大概率为当前城市分布下的全局最优解。

在运行耗时上, 模拟退火算法达到最优解仅需约 0.1 秒, 且大多数参数组合耗时均在 1 秒以内, 而遗传算法则需要 3 秒以上, 某些参数组合甚至达到了 10 秒以上。其根本原因在于算法结构: 模拟退火算法每次迭代仅维护一条路线, 且路径长度差的计算时间复杂度为 $O(1)$; 而遗传算法的每一代需要对整个种群中的 50 条路线进行交叉、变异及适应度计算, 计算开销巨大。

因此, 针对 30 座城市这种中小规模的组合优化问题, 模拟退火算法在计算性价比上优于遗传算法。

代码见 TSP.ipynb 文件, 计算结果展示见 SA.mp4/GA.mp4/SA.gif/GA.gif 文件。